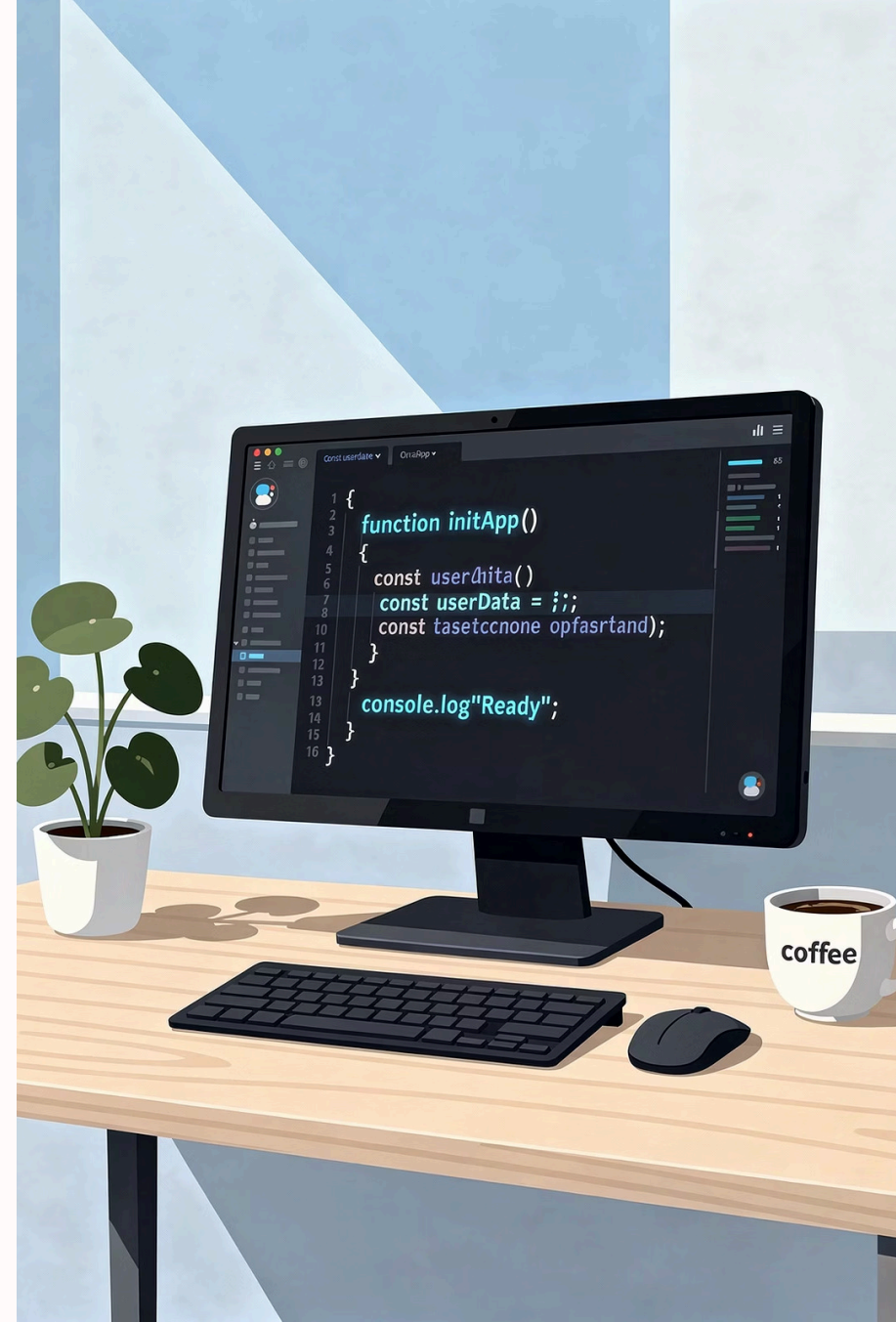


Estruturas de Repetição em JavaScript

Dominando For e While

Uma jornada técnica e prática pelos laços de repetição mais utilizados no desenvolvimento JavaScript moderno.



Roteiro da Apresentação

O que vamos aprender hoje?

01

Introdução aos Loops

Por que laços são fundamentais na automação de processos

02

O Laço For

Anatomia, sintaxe e exemplos com arrays

03

O Laço While

Controle dinâmico e condições de parada

04

Comparação & Boas Práticas

Quando usar cada um e como escrever código seguro

05

Exercício Prático

Fixação do conteúdo com desafio de código

O que são Loops e por que importam?

Loops — ou **laços de repetição** — são estruturas que permitem executar um bloco de código múltiplas vezes sem a necessidade de reescrevê-lo. Eles são a espinha dorsal da automação em programação.

Automação

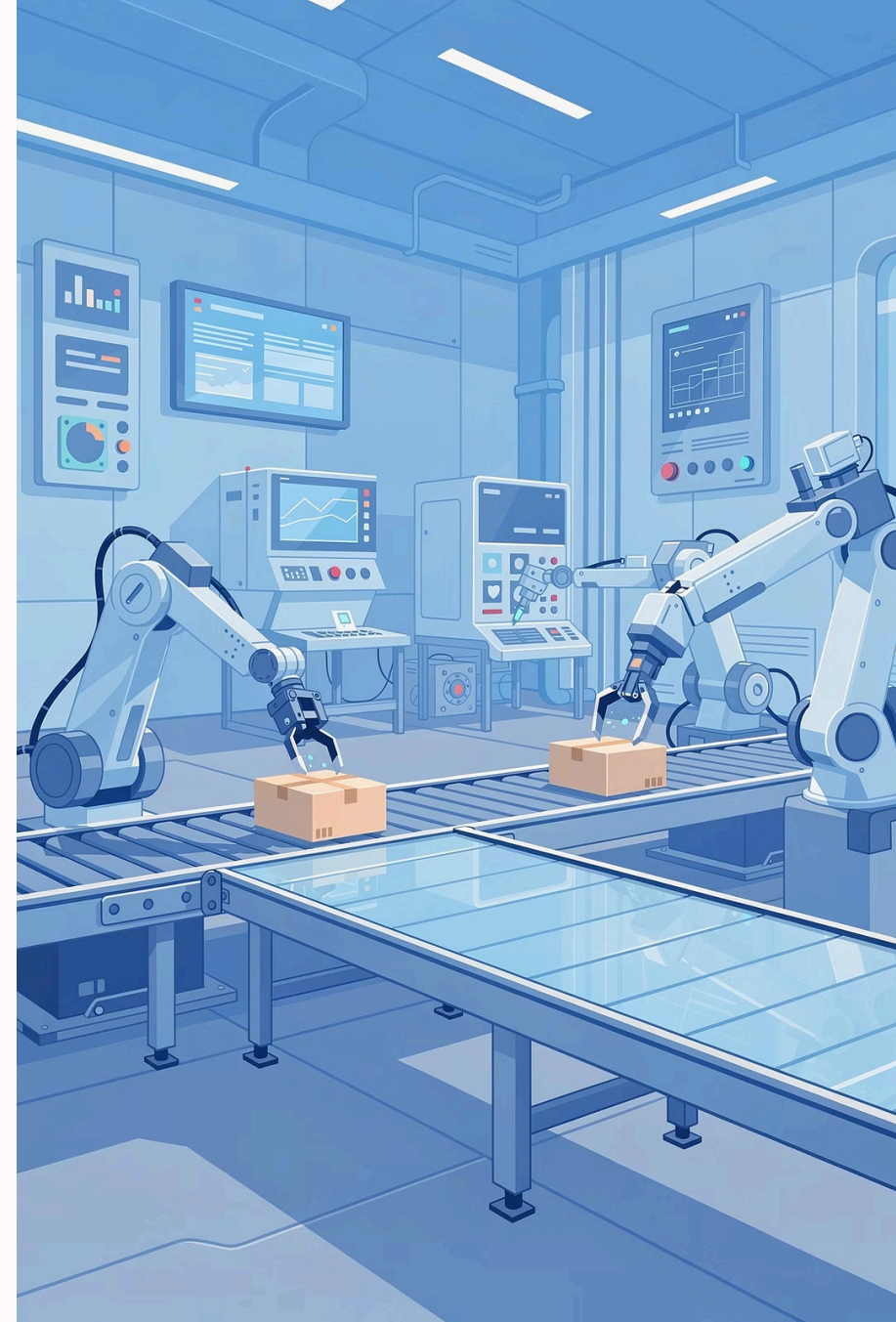
Execute tarefas repetitivas instantaneamente, sem esforço manual

Manipulação de Dados

Percorra arrays, listas e objetos com elegância e eficiência

Performance

Reduza linhas de código e melhore a manutenibilidade do projeto



Sem Loops vs. Com Loops

O Problema que os Loops Resolvem

❌ Sem Loop – Código repetitivo

```
console.log("Item 1");  
console.log("Item 2");  
console.log("Item 3");  
console.log("Item 4");  
console.log("Item 5");  
// imagine 100 itens...
```

Inviável para grandes volumes de dados. Difícil de manter e escalar.

✅ Com Loop – Elegante e escalável

```
for (let i = 1; i <= 100; i++) {  
  console.log("Item " + i);  
}  
// 100 itens em 3 linhas!
```

Flexível, legível e pronto para qualquer volume de dados. Isso é o poder dos loops.

O Laço For: Anatomia Completa

O `for` é o loop mais utilizado quando sabemos **exatamente quantas vezes** queremos iterar. Ele concentra todo o controle da iteração em uma única linha, tornando o código compacto e legível.

```
for ( inicialização ; condição ; incremento ) {  
    // bloco de código executado a cada iteração  
}
```

1 Inicialização

`let i = 0` Executada **uma vez** antes do loop iniciar. Define a variável de controle.

2 Condição

`i < 10` Verificada **antes de cada iteração**. Se falsa, o loop encerra.

3 Incremento

`i++` Executado **ao final de cada iteração**. Avança o contador.

For em Ação: Percorrendo um Array

Um dos casos de uso mais comuns do `for` é percorrer os elementos de um array. Veja como acessar cada item de forma estruturada e segura usando o índice e o `.length` do array.

```
const frutas = ["maçã", "banana", "laranja", "uva", "manga"];
```

```
for (let i = 0; i < frutas.length; i++) {  
  console.log(`Fruta ${i + 1}: ${frutas[i]}`);  
}
```

```
// Saída:
```

```
// Fruta 1: maçã
```

```
// Fruta 2: banana
```

```
// Fruta 3: laranja
```

```
// Fruta 4: uva
```

```
// Fruta 5: manga
```

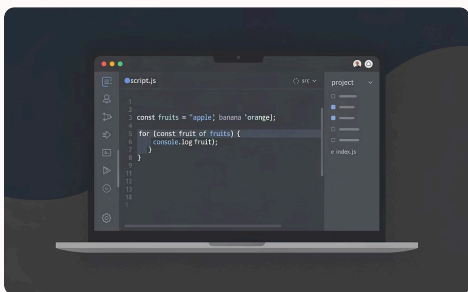


Dica Pro: Usar `frutas.length` na condição (em vez de um número fixo) torna o loop automaticamente adaptável ao tamanho do array — mesmo que ele mude dinamicamente.

Variações do For

Além do For Clássico: For...of e For...in

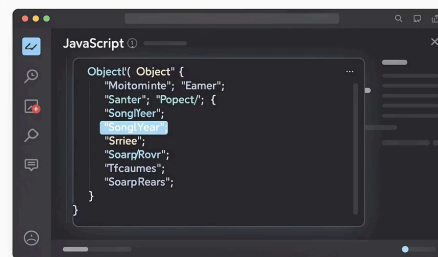
JavaScript oferece variações modernas e mais expressivas do loop `for`, cada uma com propósito distinto. Conhecê-las amplia seu arsenal de ferramentas.



For...of

```
const frutas = ["maça", "banana", "mamão"];
for (const fruta of frutas) {
  console.log(fruta);
}
```

Ideal para **arrays e iteráveis**. Mais limpo e direto — sem índice manual.



For...in

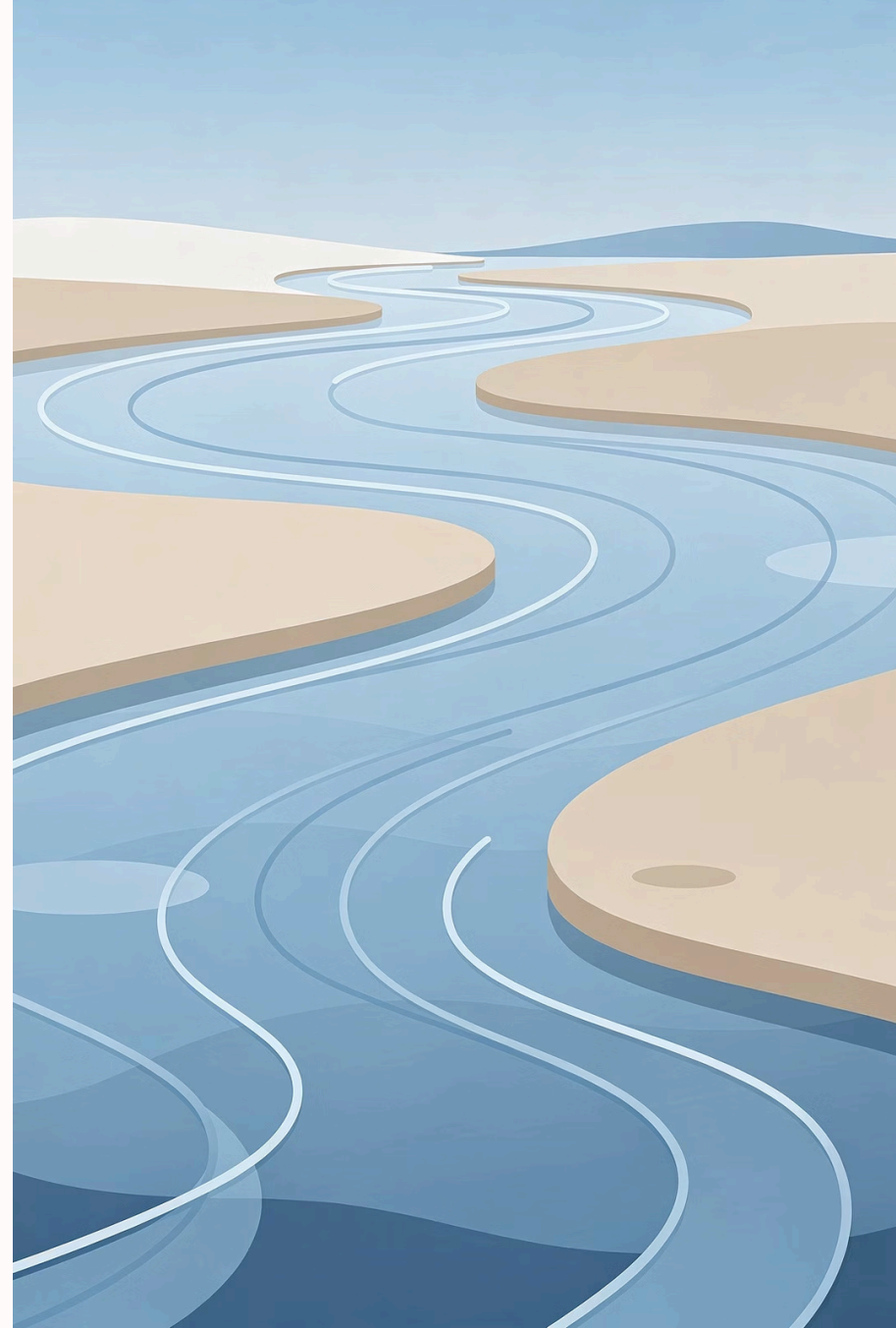
```
const pessoa = {nome:"Ana", idade:25};
for (const chave in pessoa) {
  console.log(chave, pessoa[chave]);
}
```

Ideal para percorrer as **propriedades de um objeto**. Retorna as chaves do objeto.

O Laço While: Quando a Incerteza Domina

O `while` é o loop ideal quando **não sabemos antecipadamente quantas iterações** serão necessárias. Ele continua executando enquanto uma condição permanecer verdadeira — perfeito para cenários dinâmicos como aguardar input do usuário, ler dados de uma API ou processar filas de eventos.

```
while ( condição ) {  
    // executa enquanto a condição for verdadeira  
    // IMPORTANTE: atualizar algo que afete a condição!  
}
```



While em Ação: Exemplo Prático

Veja um exemplo real de uso do `while`: simulando um processo de saque bancário onde o loop continua enquanto houver saldo disponível para a operação solicitada.

```
let saldo = 500;
const valorSaque = 100;

while (saldo >= valorSaque) {
  saldo -= valorSaque;
  console.log(`Saque realizado. Saldo restante: R${saldo}`);
}

console.log("Saldo insuficiente para novo saque.");

// Saque realizado. Saldo restante: R$400
// Saque realizado. Saldo restante: R$300
// ...até R$0
```

📌 ⚠️ **Atenção ao Loop Infinito:** Perceba que `saldo -= valorSaque` modifica a variável de controle a cada iteração. Sem essa linha, `saldo` nunca mudaria, a condição seria sempre verdadeira e o navegador travaria.

O Perigo do Loop Infinito

Um loop infinito ocorre quando a condição de parada **nunca se torna falsa**. O resultado é o travamento completo do navegador ou da aplicação. Veja a diferença entre um código perigoso e um seguro:

Loop Infinito – PERIGO

```
let contador = 0;

while (contador < 10) {
  console.log("Executando...");
  // contador nunca muda!
  // Navegador trava!
```



A variável `contador` nunca é incrementada — a condição jamais se torna falsa.

While Seguro – Correto

```
let contador = 0;

while (contador < 10) {
  console.log("Iteração: " + contador);
  contador++; // condição evolui!
  // Para após 10 execuções
```



Incrementar `contador` garante que a condição eventualmente se torne falsa.

Do...While: O Irmão Garantido

O `do...while` é uma variação do `while` que garante que o bloco de código seja executado **pelo menos uma vez**, independentemente da condição — pois a verificação ocorre apenas ao final da iteração.

```
let tentativas = 0;

do {
  console.log(`Tentativa ${tentativas + 1} de conexão...`);
  tentativas++;
} while (tentativas < 3);

// Tentativa 1 de conexão...
// Tentativa 2 de conexão...
// Tentativa 3 de conexão...
```

Caso de Uso Clássico

Validação de formulários onde precisamos exibir o campo ao usuário ao menos uma vez antes de verificar se o input é válido.

Diferença do While

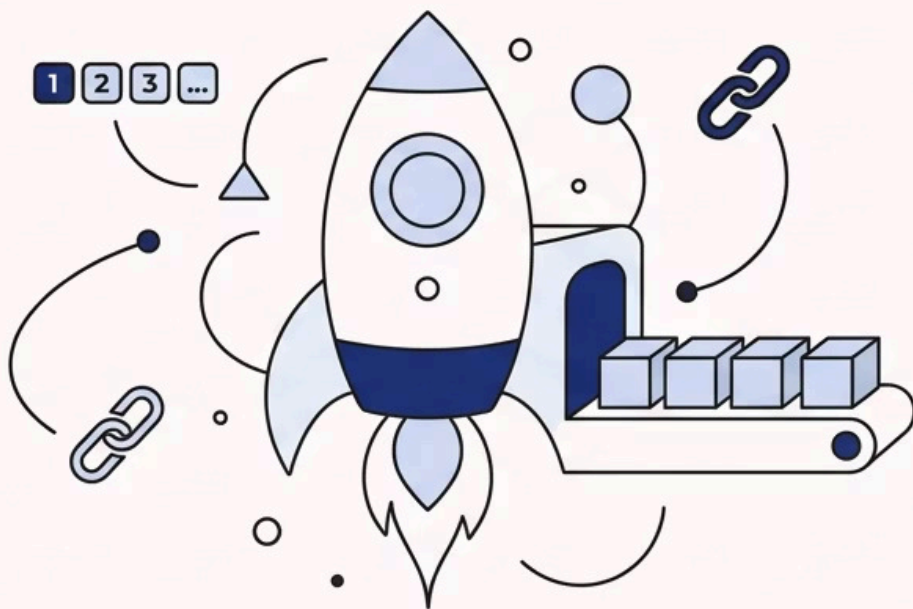
No `while`, se a condição for falsa de início, o bloco nunca executa.
No `do...while`, executa ao menos uma vez.

For vs. While: Comparação Visual

Entender quando usar cada loop é tão importante quanto saber a sintaxe. A tabela abaixo resume as principais diferenças para guiar sua decisão:

Critério	 For	 While
Número de iterações	Conhecido previamente	Desconhecido / dinâmico
Controle	Centralizado em uma linha	Distribuído no corpo do loop
Uso típico	Arrays, índices, contagem fixa	Eventos, APIs, inputs do usuário
Previsibilidade	Alta — comportamento fixo	Variável — depende de dados externos
Risco de loop infinito	Baixo — incremento é explícito	Alto — requer atenção à condição
Legibilidade	Muito clara para iterações fixas	Clara para lógicas condicionais

FOR LOOP: Previsível e baseado em contagem. Ideal para arrays e listas.



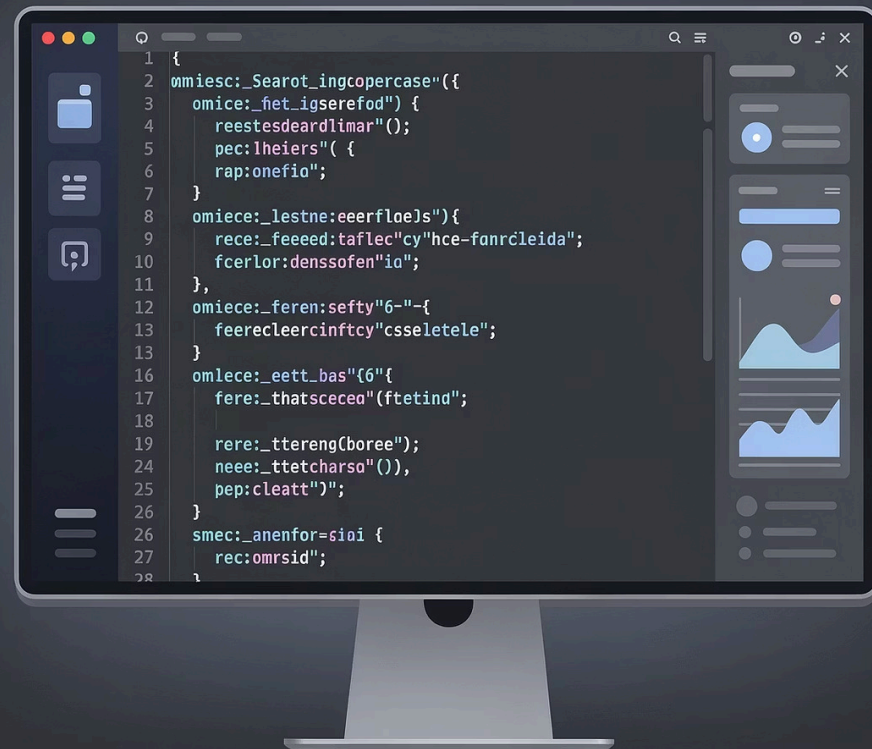
WHILE LOOP: Dinâmico e baseado em condição. Ideal para eventos e inputs.



Use o `for` quando souber a quantidade de iterações. Use o `while` quando a condição de parada depender de fatores externos ou dinâmicos.

Boas Práticas com Loops em JavaScript

Escrever loops funcionais é o primeiro passo. Escrever loops **eficientes, seguros e legíveis** é o que diferencia um desenvolvedor júnior de um sênior. Siga estas diretrizes essenciais:



Boas Práticas: Código e Performance

→ Obtenha o cache do length do array

Em arrays grandes, calcular `array.length` a cada iteração tem custo. Prefira: `const len = arr.length; for(let i=0; i < len; i++)` para otimizar performance.

→ Prefira métodos funcionais quando possível

`forEach`, `map`, `filter` e `reduce` são mais expressivos para manipulação de arrays. Reserve os loops clássicos para lógica de controle complexa.

→ Sempre garanta uma condição de saída

No `while`, certifique-se de que a variável de controle é modificada dentro do loop. Use `break` como válvula de segurança em casos extremos.

→ Nomeie variáveis de controle com clareza

Em vez de apenas `i`, use nomes como `indexProduto` ou `posicao` quando o contexto for complexo. Código legível é código mantível.

Break e Continue: Controlando o Fluxo

Duas palavras-chave essenciais para controle avançado dentro de loops. Usá-las corretamente evita processamento desnecessário e torna o código mais eficiente.

Break – Interrompe o loop

```
for (let i = 0; i < 10; i++) {  
  if (i === 5) break;  
  console.log(i);  
}  
// Exibe: 0, 1, 2, 3, 4  
// Para completamente em i=5
```

Use para encerrar o loop assim que encontrar o resultado desejado.

Continue – Pula para próxima iteração

```
for (let i = 0; i < 6; i++) {  
  if (i === 3) continue;  
  console.log(i);  
}  
// Exibe: 0, 1, 2, 4, 5  
// Pula apenas o i=3
```

Use para ignorar casos específicos sem encerrar o loop inteiro.

Qual Escolher? Guia de Decisão

A escolha entre `for` e `while` deve ser guiada pelo contexto do problema, não por preferência pessoal. Use este guia rápido para tomar a decisão certa:



Na prática, o `for` cobre a maioria dos casos com arrays. O `while` brilha em cenários reativos e dinâmicos onde o número de iterações é incerto.

Conclusão: Dominando os Loops

Use For quando...

Souber o número de iterações, percorrer arrays e coleções, ou precisar de controle de índice preciso.

Use While quando...

A condição de parada for dinâmica, depender de input externo, evento ou dados assíncronos.

Sempre lembre...

Garantir condição de saída, nomear variáveis com clareza e preferir métodos funcionais para código mais expressivo.

"Código limpo não é aquele que o computador entende — é aquele que **outros desenvolvedores conseguem ler e manter com facilidade.**"

Exercício Prático

Hora de Codar: Desafio de Fixação

Coloque em prática tudo que aprendemos! Resolva os três desafios abaixo no seu editor favorito. Tente sem consultar o material antes de conferir a dica.

1

Tabuada com For

Crie um loop `for` que imprima a tabuada do número 7 (de 1 a 10) no formato: "7 x 1 = 7".

2

Senha com While

Use um `while` para simular tentativas de login. Defina uma senha correta e continue pedindo até o usuário acertar (use um array de tentativas para simular).

3

Filtro com Continue

Percorra o array `[1,2,3,4,5,6,7,8,9,10]` com `for` e use `continue` para exibir apenas os números **ímpares**.